

Support Vector Machines (SVM)

Francisco Rodrigues

Instituto de Ciências Matemáticas e de Computação

Universidade de São Paulo

francisco@icmc.usp.br

SVM

- Imaginem que temos duas classes linearmente separáveis.
- Uma questão importante é: qual reta melhor separa as classes?



SVM

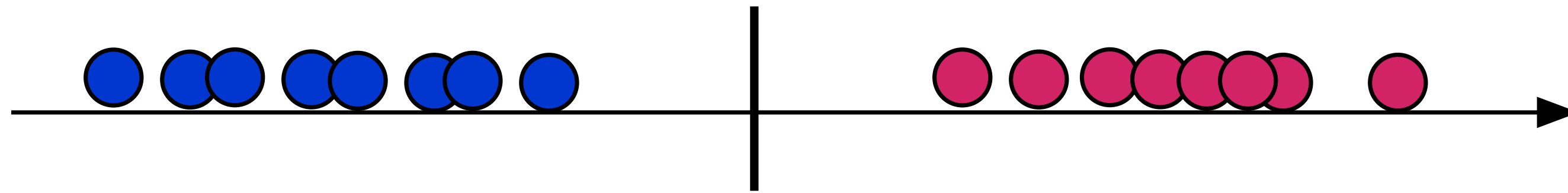
- Vamos considerar o caso unidimensional.



Imaginem que medimos um certo atributo,
como o nível de glicose de pacientes.

SVM

- Caso unidimensional.



Em azul temos o valor para pacientes saudáveis e, em vermelho, para pessoas que são diabéticas.

SVM

- Caso unidimensional.



Baseando-se nas observações, definimos um limiar de separação entre as classes.

SVM

- Caso unidimensional.



Esse limiar é equidistante aos pontos de fronteira de cada classe.

SVM

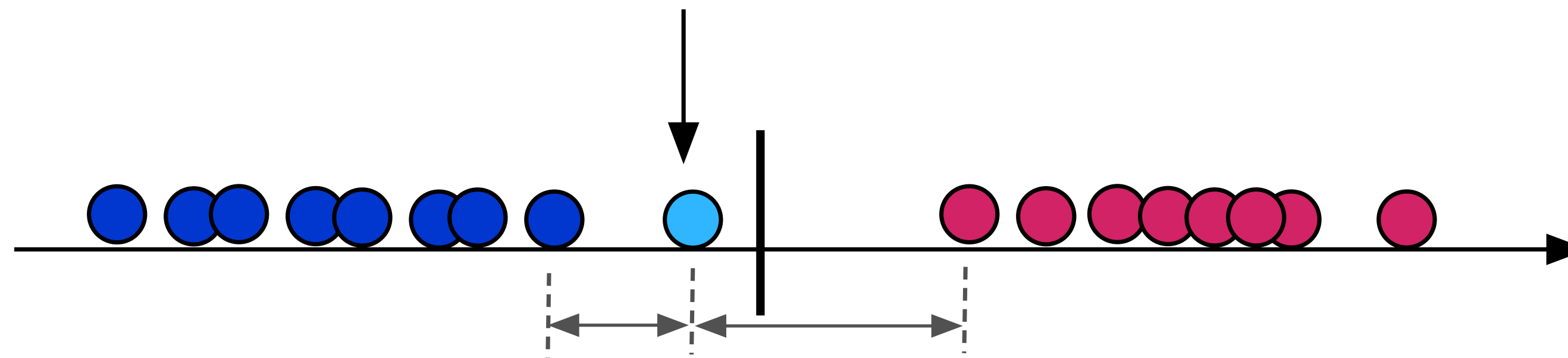
- Caso unidimensional.



Temos uma nova observação, que tem nível de glicose menor do que o limiar.
Logo, classificamos como saudável.

SVM

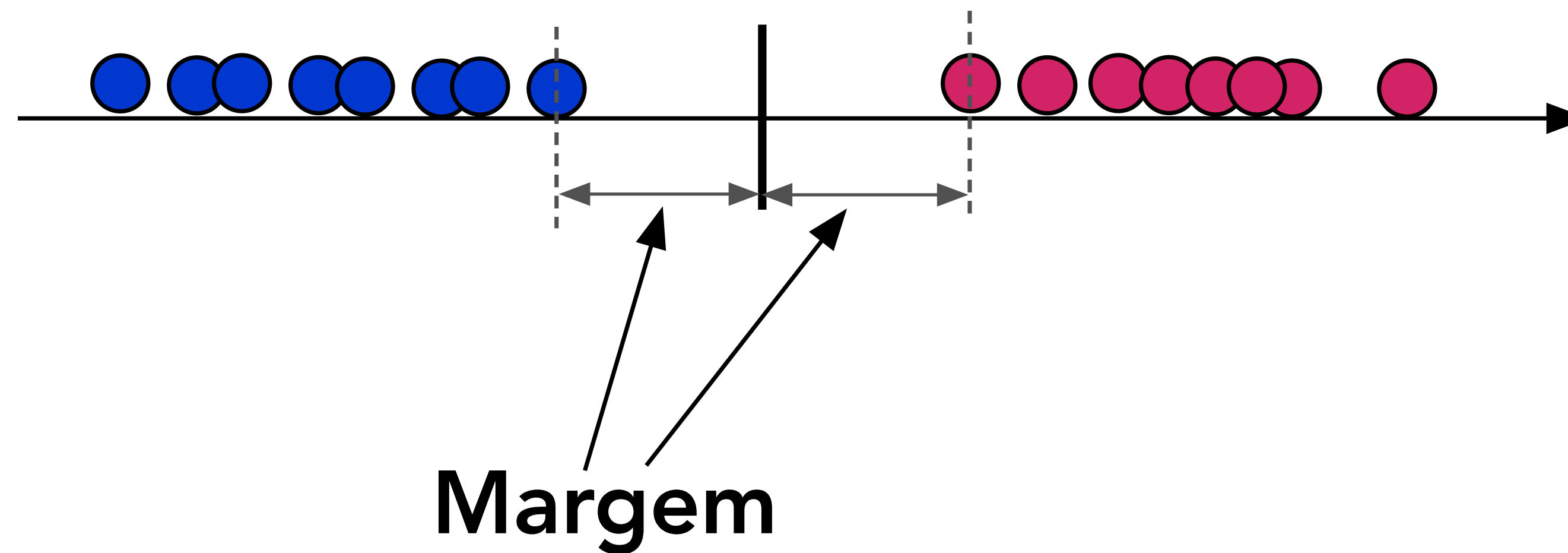
- Caso unidimensional.



De fato, a observação está mais próxima da classe em azul.

SVM

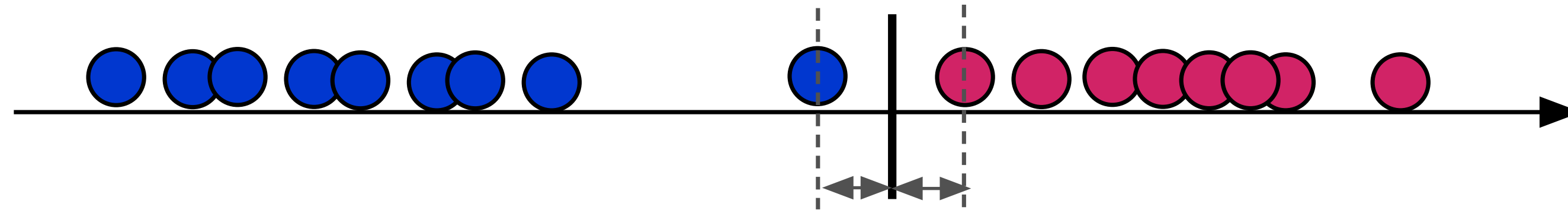
- Caso unidimensional.



Maximum Margin Classifier

SVM

- Caso unidimensional.



Maximum Margin Classifier

Imagine que agora que temos um outlier da classe em azul.

SVM

- Caso unidimensional.



Maximum Margin Classifier

O limiar é movido para outro lugar, com margem muito mais estreita.

SVM

- Caso unidimensional.



Maximum Margin Classifier: sensíveis a outliers!

Temos uma nova observação, que será classificada como da classe em azul.

SVM

- Caso unidimensional.



Maximum Margin Classifier: sensíveis a outliers!

Mas note que ela está mais próxima da classe em vermelho!

SVM

- Caso unidimensional.



Logo, vemos que usar uma margem que é a média da distância entre os pontos de fronteira é sensível a outliers.

SVM

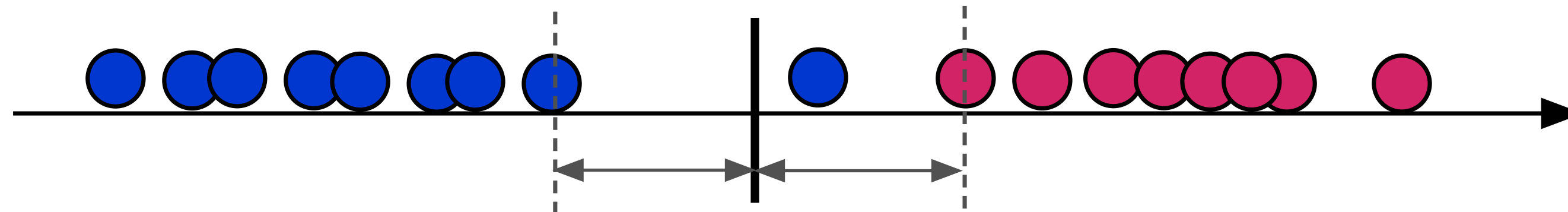
- Caso unidimensional.



Para obter um classificador robusto a outliers,
vamos permitir que erros possam ser cometidos na classificação.

SVM

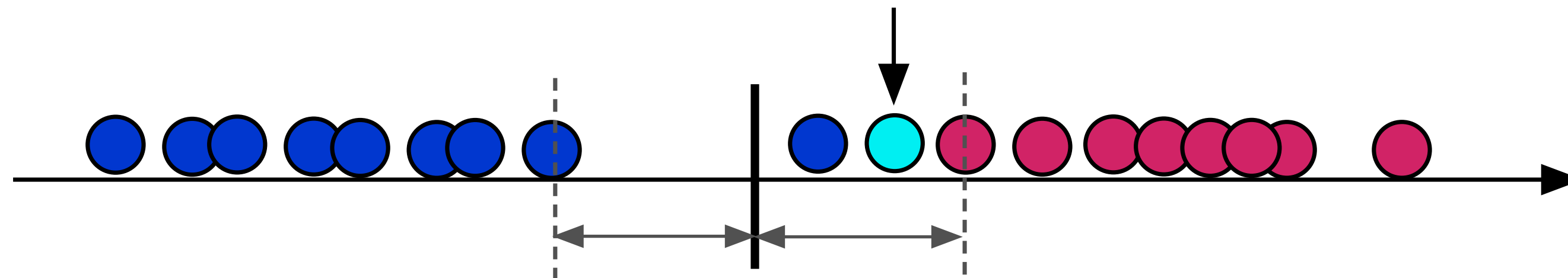
- Caso unidimensional.



Assim, movemos a margem.

SVM

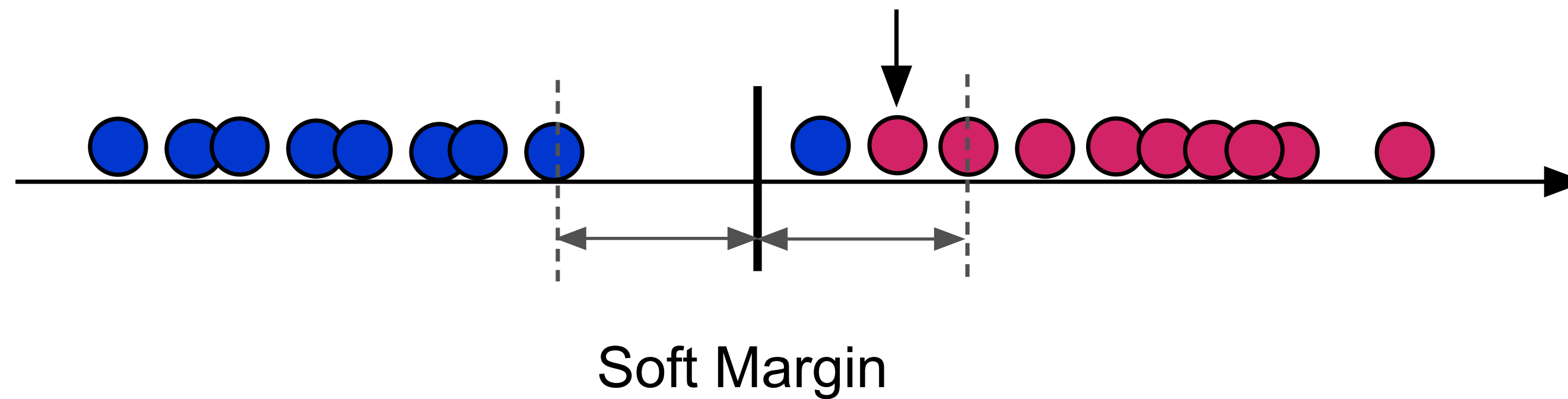
- Caso unidimensional.



Com essa nova fronteira.

SVM

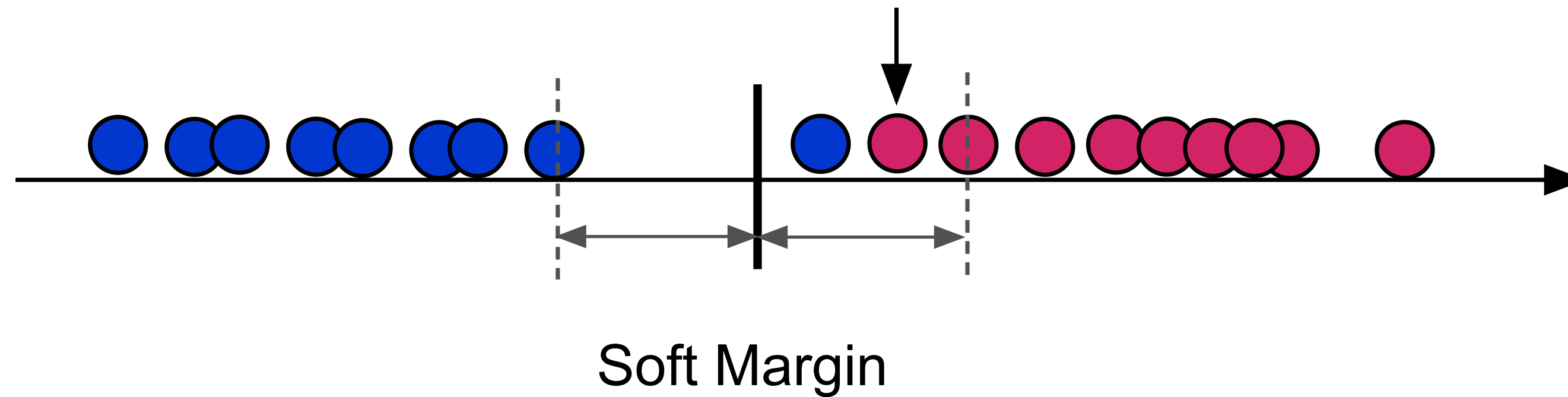
- Caso unidimensional.



A nova observação é classificada como vermelha, que faz mais sentido.

SVM

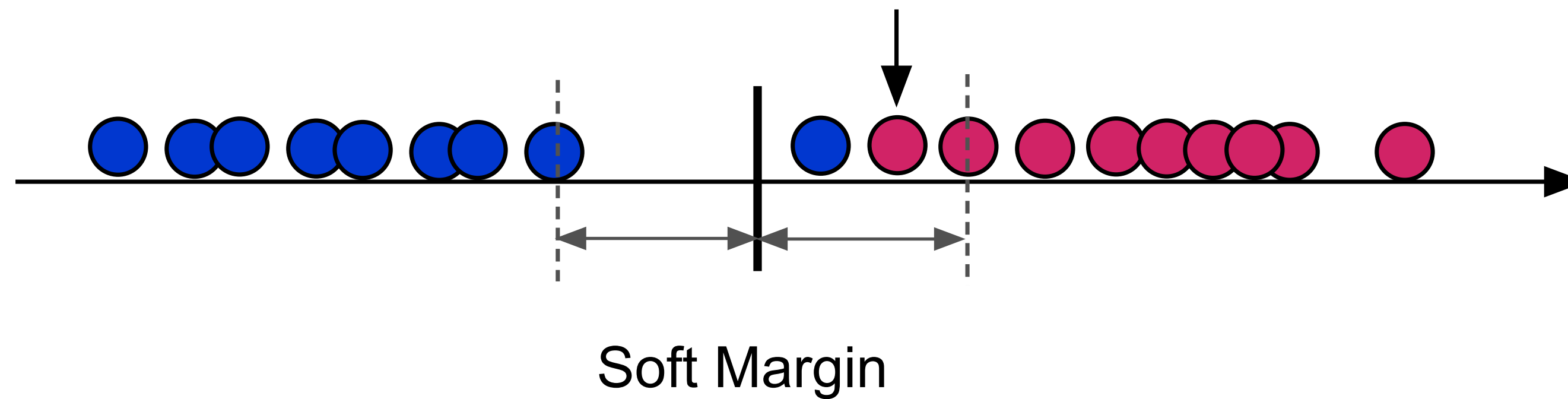
- Caso unidimensional.



Escolher a posição da margem é um exemplo do equilíbrio entre viés e variância.

SVM

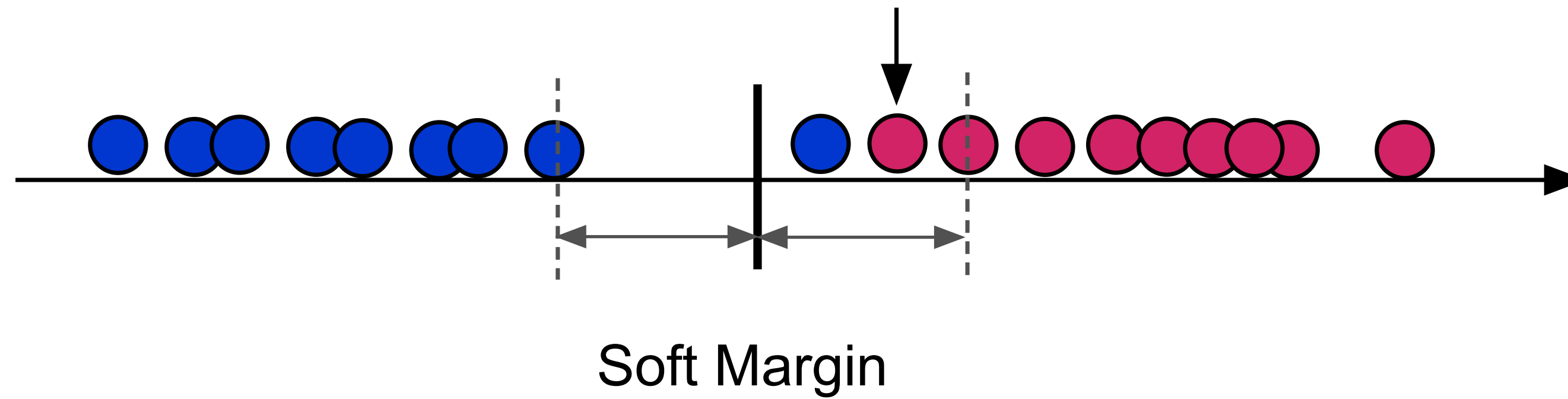
- Caso unidimensional.



Margem muito estreita pode levar a overfitting.
Margem muito larga, a underfitting.

SVM

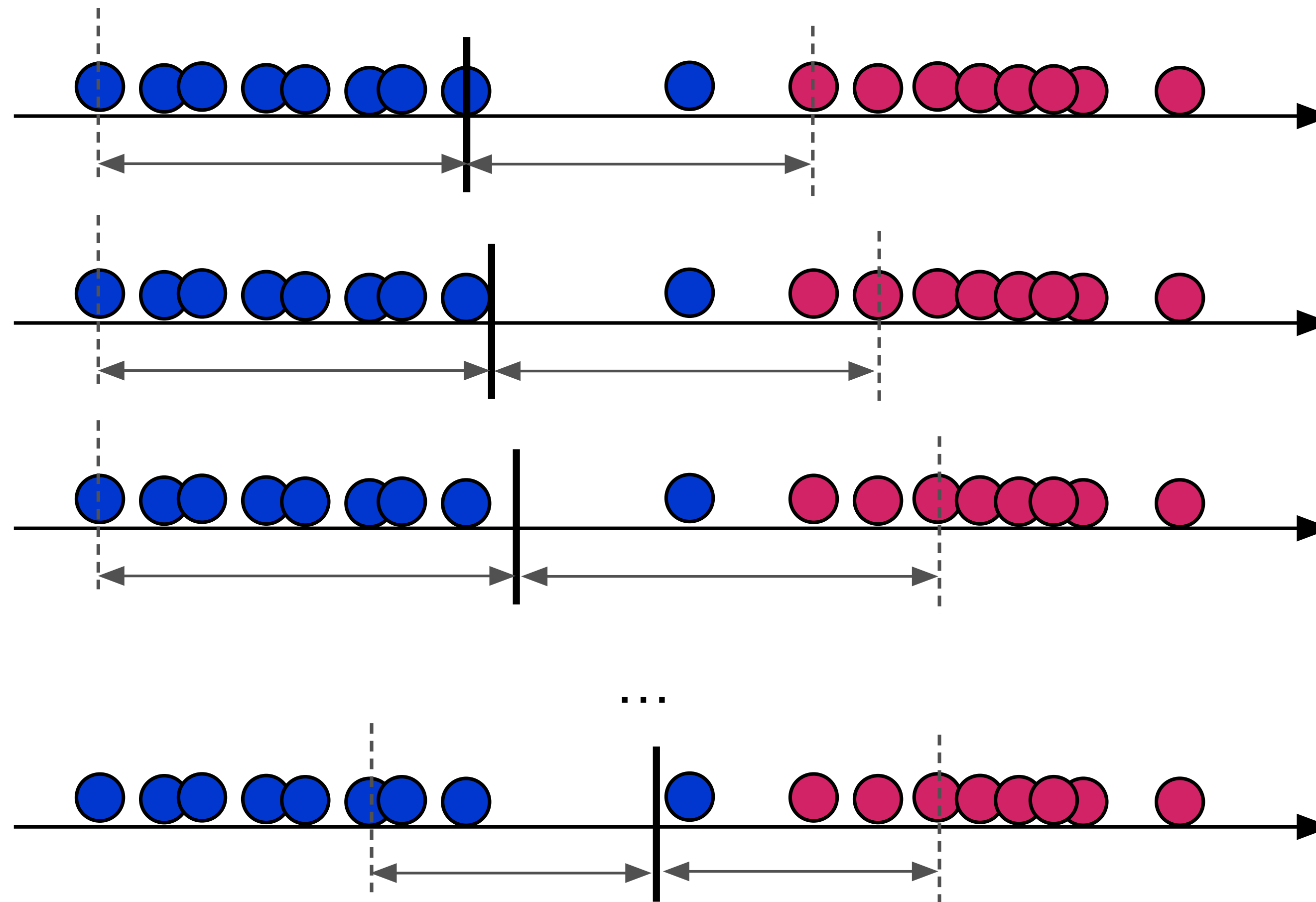
- Caso unidimensional.



Mas com encontrar a melhor margem?

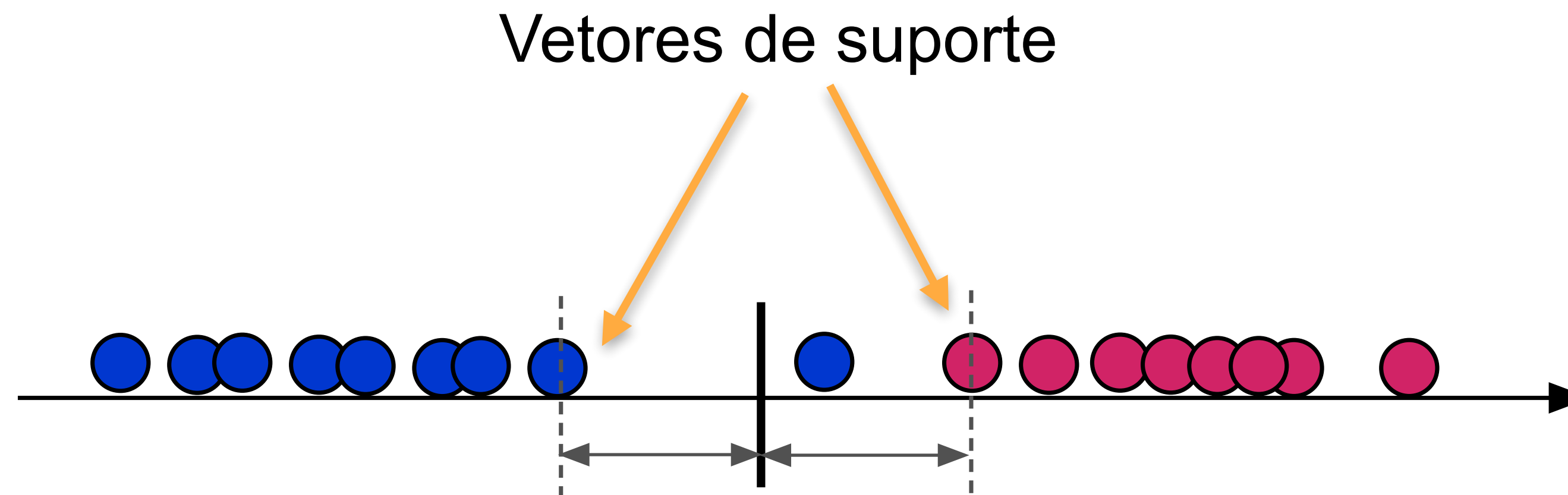
SVM

Para encontrar a melhor margem, usamos validação cruzada.



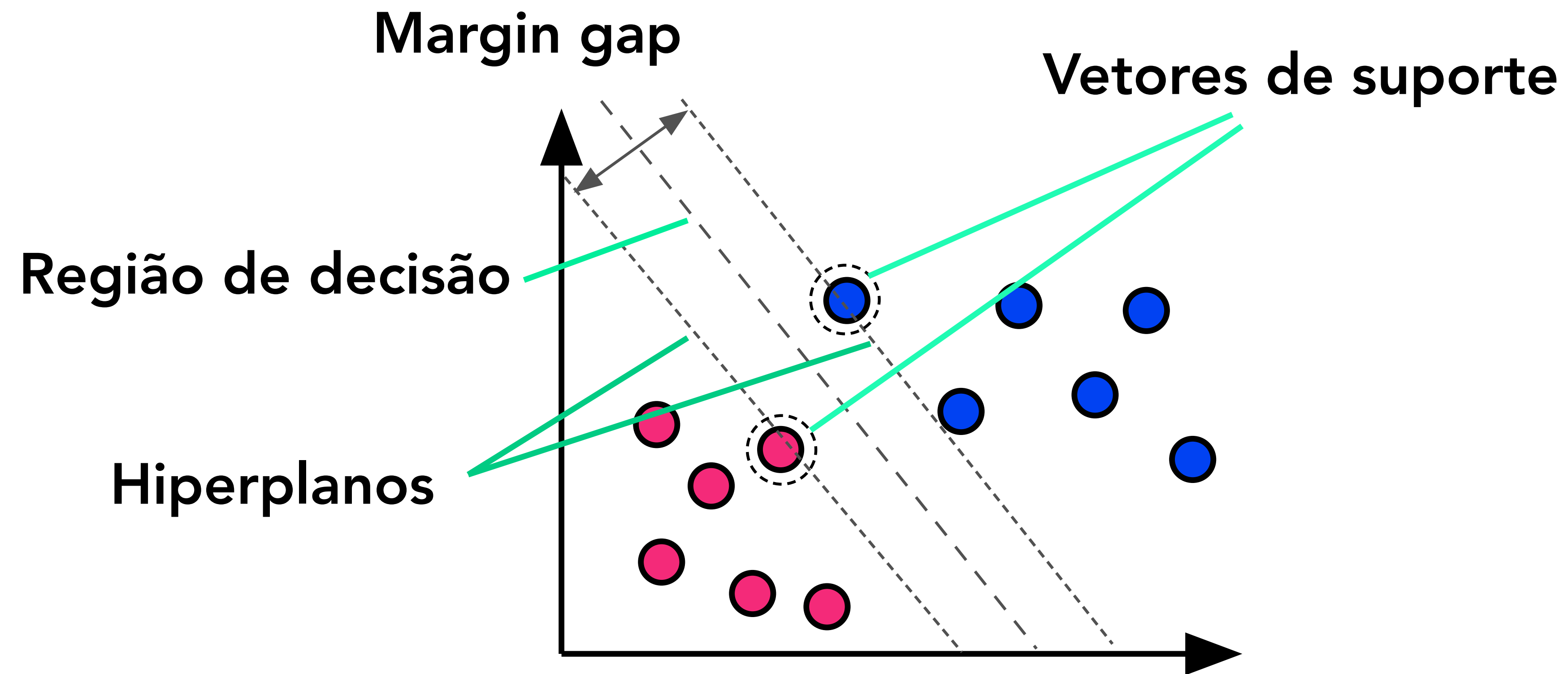
SVM

Esse método é chamado **classificador com vetores de suporte**.



SVM

Em duas dimensões:



Classificador SVM (Support Vector Machine):

- É um algoritmo de **aprendizado supervisionado** utilizado para classificação e, em alguns casos, para regressão.
- Ele busca encontrar uma **hiperplano de separação** que maximize a **margem** entre diferentes classes em um conjunto de dados.
- ***A margem é a distância entre o hiperplano e os pontos mais próximos de cada classe, conhecidos como vetores de suporte.***

Classificador SVM (Support Vector Machine):

- O objetivo principal do SVM é **otimizar** esse hiperplano para garantir que os dados de diferentes classes sejam separados da melhor maneira possível.
- Em casos em que os dados não são linearmente separáveis, o SVM pode usar um truque chamado ***kernel trick***, que transforma os dados em um espaço de dimensões mais altas, onde uma separação linear seja possível.

SVM

Componentes principais de um classificador SVM:

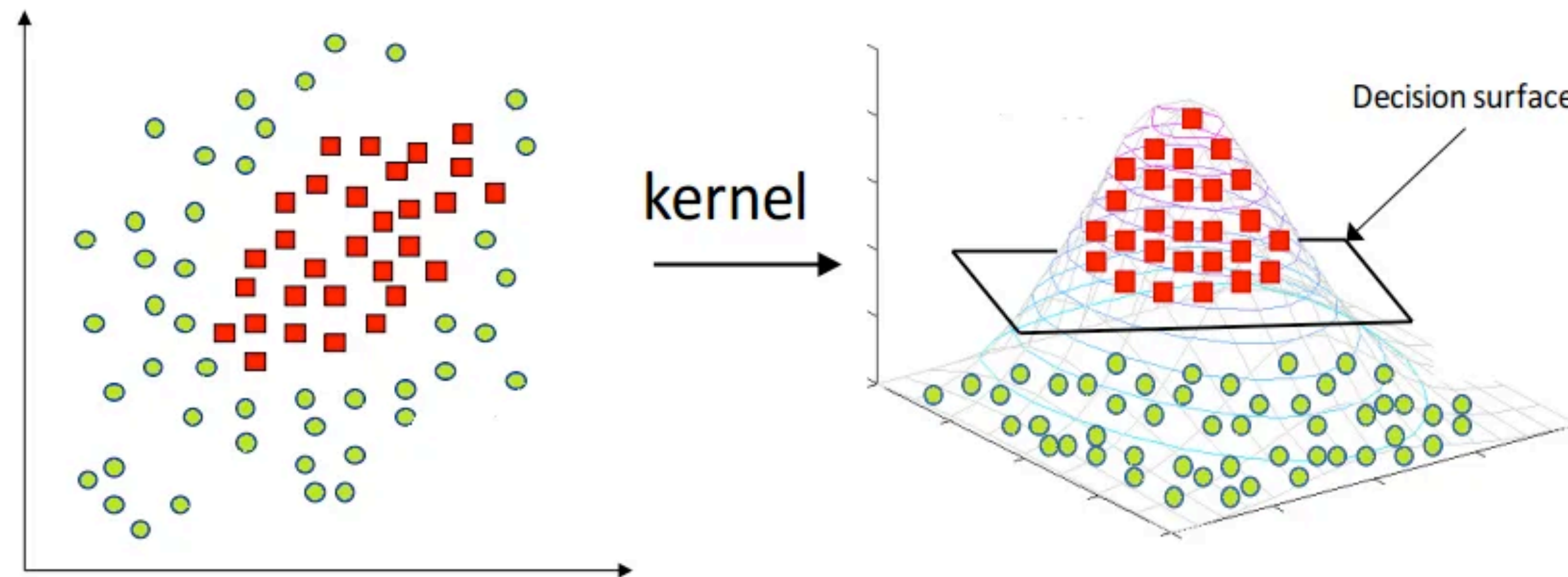
- **Hiperplano de Separação:** Uma linha (em 2D), plano (em 3D) ou uma forma mais geral (em maiores dimensões) que separa os dados de diferentes classes.
- **Vetores de Suporte:** São os pontos de dados mais próximos do hiperplano de separação. Eles são essenciais para a construção do modelo, pois o SVM tenta maximizar a margem entre as classes com base nesses pontos.
- **Kernel:** Uma função matemática que permite que o SVM trabalhe com dados não linearmente separáveis, projetando-os para um espaço de maior dimensão onde possam ser linearmente separados.



SVM

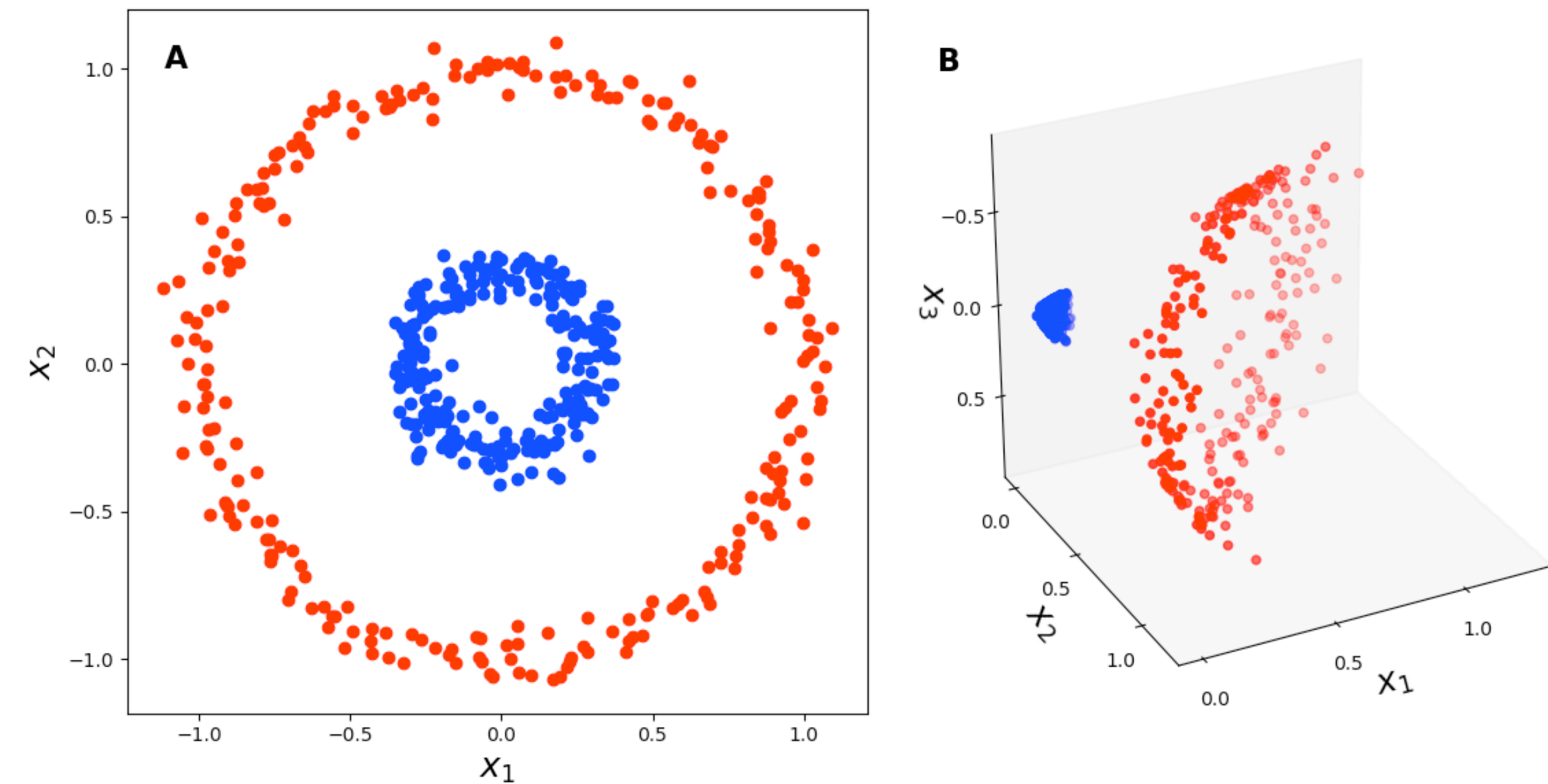
Problema: Classes não linearmente separáveis:

Solução: Kernel trick!

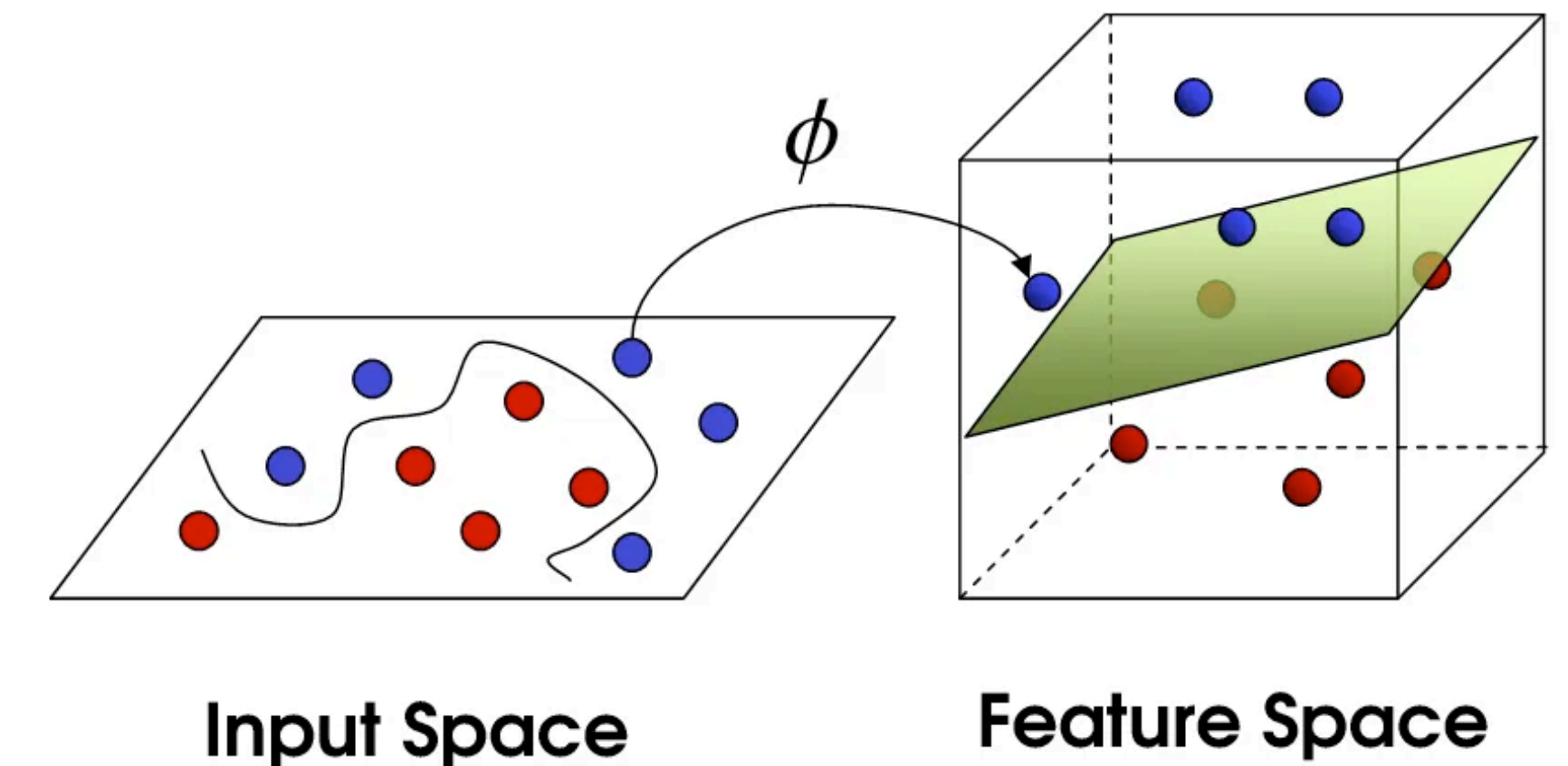
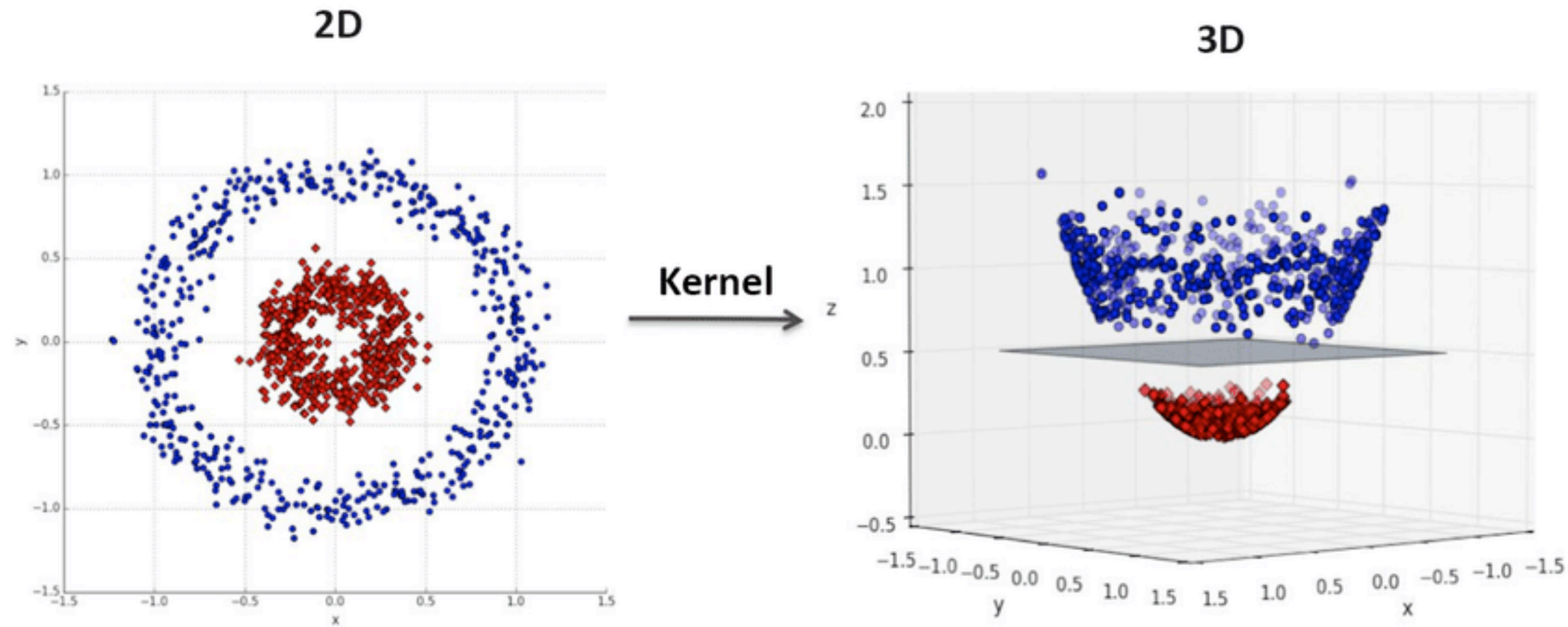


SVM

- O **kernel trick** é uma técnica usada no algoritmo SVM para lidar com problemas em que os dados não são linearmente separáveis.
- O objetivo é permitir que o SVM encontre um hiperplano de separação **em um espaço de características mais alto**, onde os dados podem ser linearmente separáveis, **sem precisar** realmente calcular as novas coordenadas dos dados nesse espaço de maior dimensão.
- Isso é feito por meio de uma função chamada **kernel**, que calcula a similaridade entre os dados em seu espaço original e no espaço transformado de maneira eficiente.



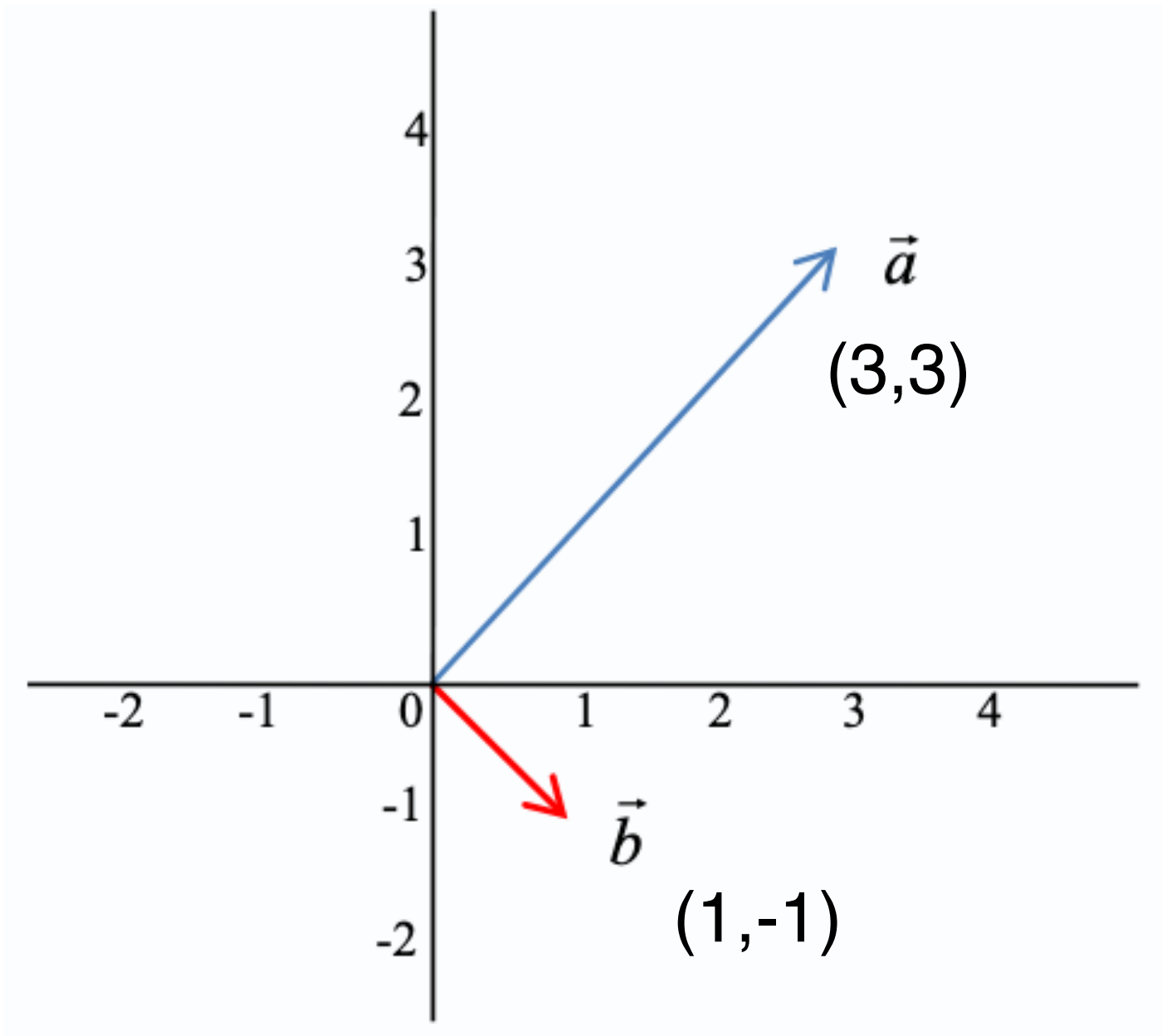
SVM



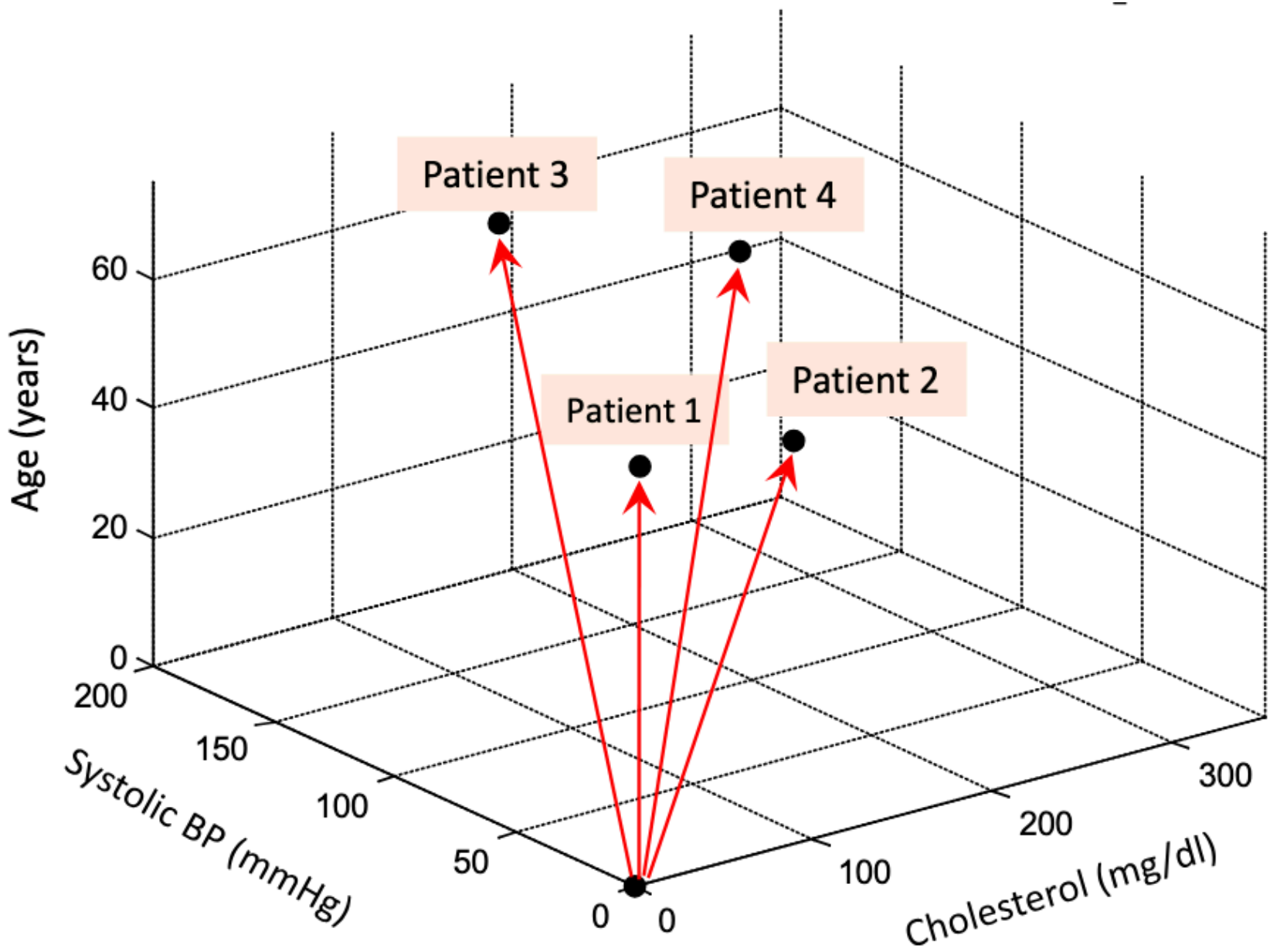
- **Linear:** $k(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i \cdot \mathbf{x}_j)$
- **Polynomial:** $k(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i \cdot \mathbf{x}_j + r)^d$.
- **Gaussian radial basis function:** $k(\mathbf{x}_i, \mathbf{x}_j) = \exp \left(-\gamma \left\| \mathbf{x}_i - \mathbf{x}_j \right\|^2 \right)$
- **Sigmóide:** $k(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\kappa \mathbf{x}_i \cdot \mathbf{x}_j + c)$

SVM

- Ideia básica:



Patient id	Cholesterol (mg/dl)	Systolic BP (mmHg)	Age (years)	Tail of the vector	Arrow-head of the vector
1	150	110	35	(0,0,0)	(150, 110, 35)
2	250	120	30	(0,0,0)	(250, 120, 30)
3	140	160	65	(0,0,0)	(140, 160, 65)
4	300	180	45	(0,0,0)	(300, 180, 45)



SVM

- Ideia básica:

$$\hat{y} = \begin{cases} 0 & \text{if } \mathbf{w}^T \mathbf{x} + b < 0, \\ 1 & \text{if } \mathbf{w}^T \mathbf{x} + b \geq 0 \end{cases}$$



SVM

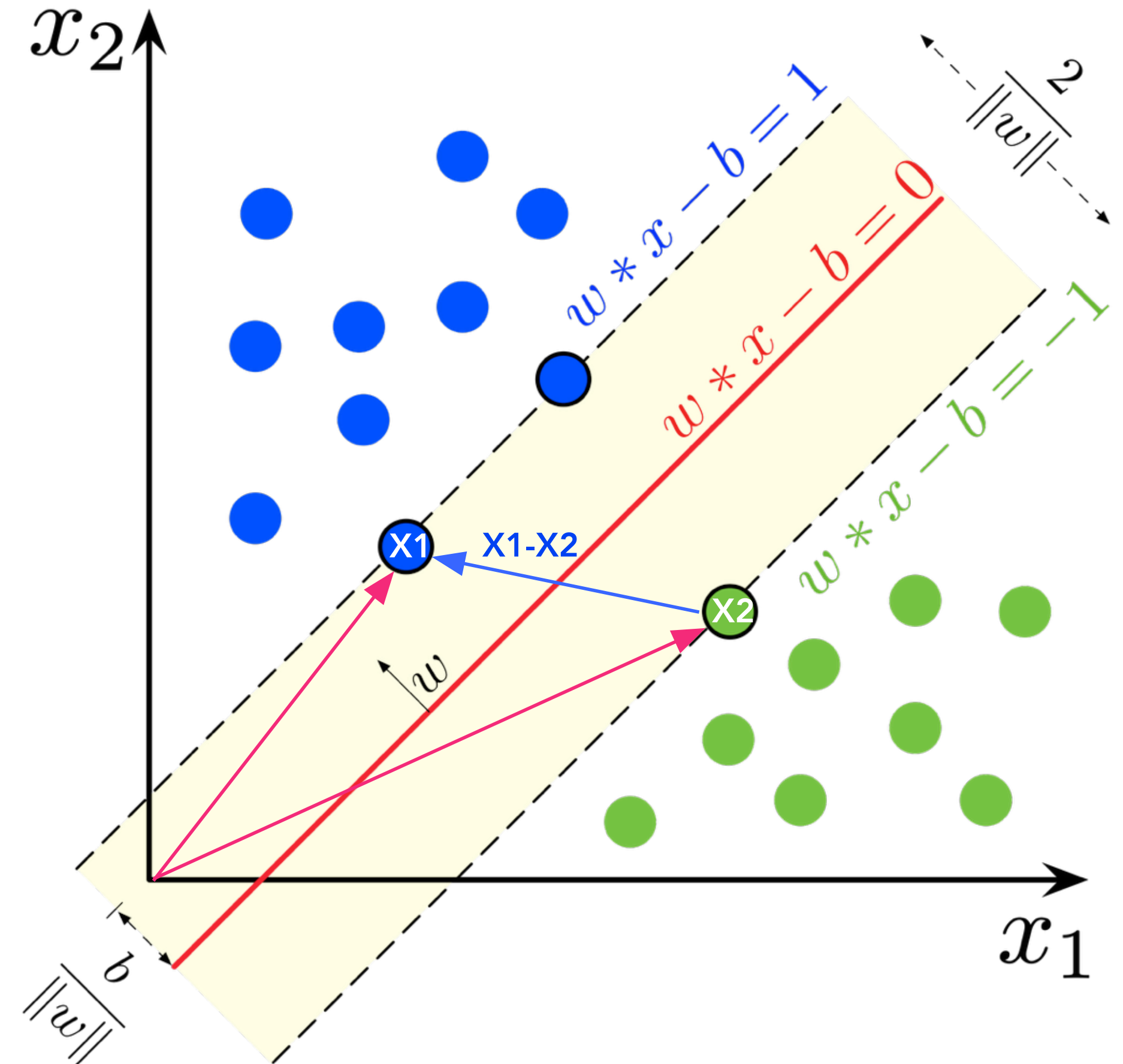
- Ideia básica:

$$\hat{y} = \begin{cases} 0 & \text{if } \mathbf{w}^\top \mathbf{x} + b < 0, \\ 1 & \text{if } \mathbf{w}^\top \mathbf{x} + b \geq 0 \end{cases}$$

$$\mathbf{w} \cdot \mathbf{x}_1 + b = 1$$

$$\mathbf{w} \cdot \mathbf{x}_2 + b = -1$$

$$\begin{aligned} \text{Largura} &= (\mathbf{x}_1 - \mathbf{x}_2) \cdot \frac{\mathbf{w}}{\|\mathbf{w}\|} \\ &= \frac{(\mathbf{x}_1 - \mathbf{x}_2) \cdot \mathbf{w}}{\|\mathbf{w}\|} \\ &= \frac{\mathbf{x}_1 \cdot \mathbf{w} - \mathbf{x}_2 \cdot \mathbf{w}}{\|\mathbf{w}\|} \\ &= \frac{1 - b - (-1 - b)}{\|\mathbf{w}\|} \\ &= \frac{2}{\|\mathbf{w}\|} \end{aligned}$$



SVM

- **Caso linear:**

A tarefa de aprendizado no SVM pode ser formalizada como o seguinte problema de

otimização com restrições:

- $$\min_w \frac{2}{\|w\|}$$

- Sujeito à: $y_i(w \cdot x_i + b) \geq 1, \quad i = 1, 2, \dots, N$



SVM

- **Caso não linear:**

A tarefa de aprendizado no SVM pode ser formalizada como o seguinte problema de otimização com restrições:

- $$\min_w \frac{2}{\|\mathbf{w}\|}$$
- Sujeito à: $y_i(\mathbf{w} \cdot \phi(\mathbf{x}_i) + b) \geq 1, \quad i = 1, 2, \dots, N$



$$k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j).$$

Função kernel

SVM

- Kernel polinomial

```
import numpy as np
from sklearn import datasets
from sklearn.pipeline import Pipeline

from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

iris = datasets.load_iris()
X = iris["data"][:, (2, 3)] # petal length, petal width
y = (iris["target"] == 2).astype(np.float64) # Iris virginica

svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("linear_svc", LinearSVC(C=1, loss="hinge")),
])

svm_clf.fit(X, y)
```



Se o seu modelo SVM leva ao overfitting, você pode tentar regularizá-lo reduzindo C .

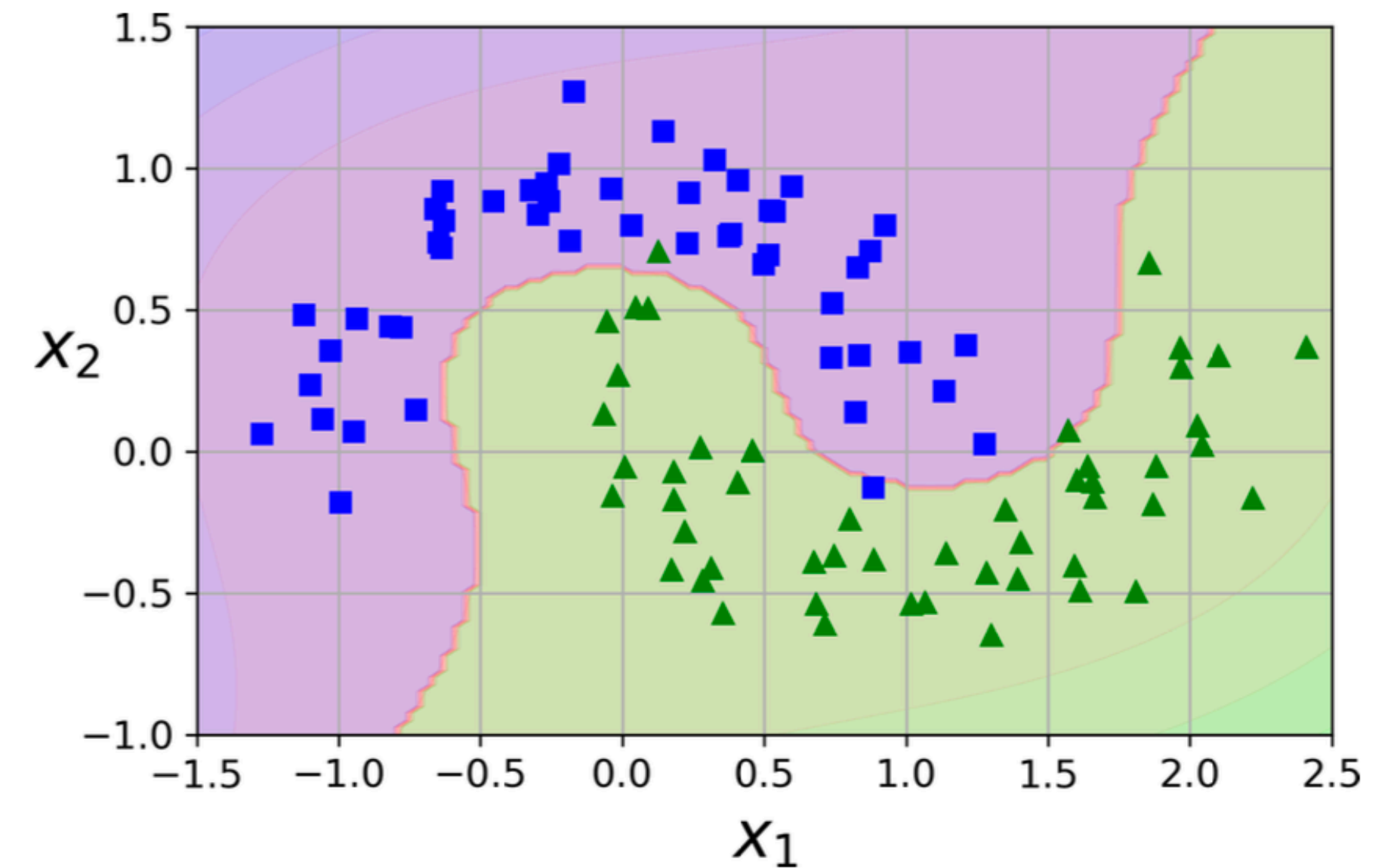
SVM

- Kernel polynomial

```
from sklearn.datasets import make_moons
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures

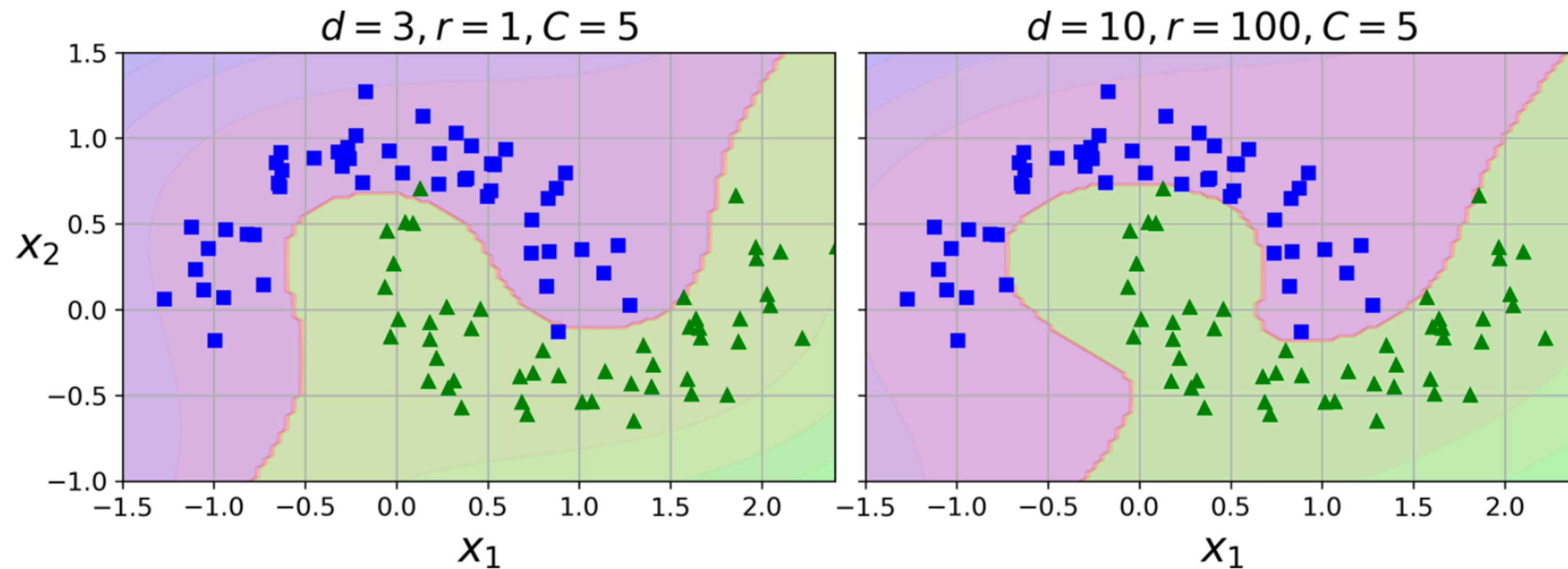
X, y = make_moons(n_samples=100, noise=0.15)
polynomial_svm_clf = Pipeline([
    ("poly_features", PolynomialFeatures(degree=3)),
    ("scaler", StandardScaler()),
    ("svm_clf", LinearSVC(C=10, loss="hinge"))
])

polynomial_svm_clf.fit(X, y)
```



SVM

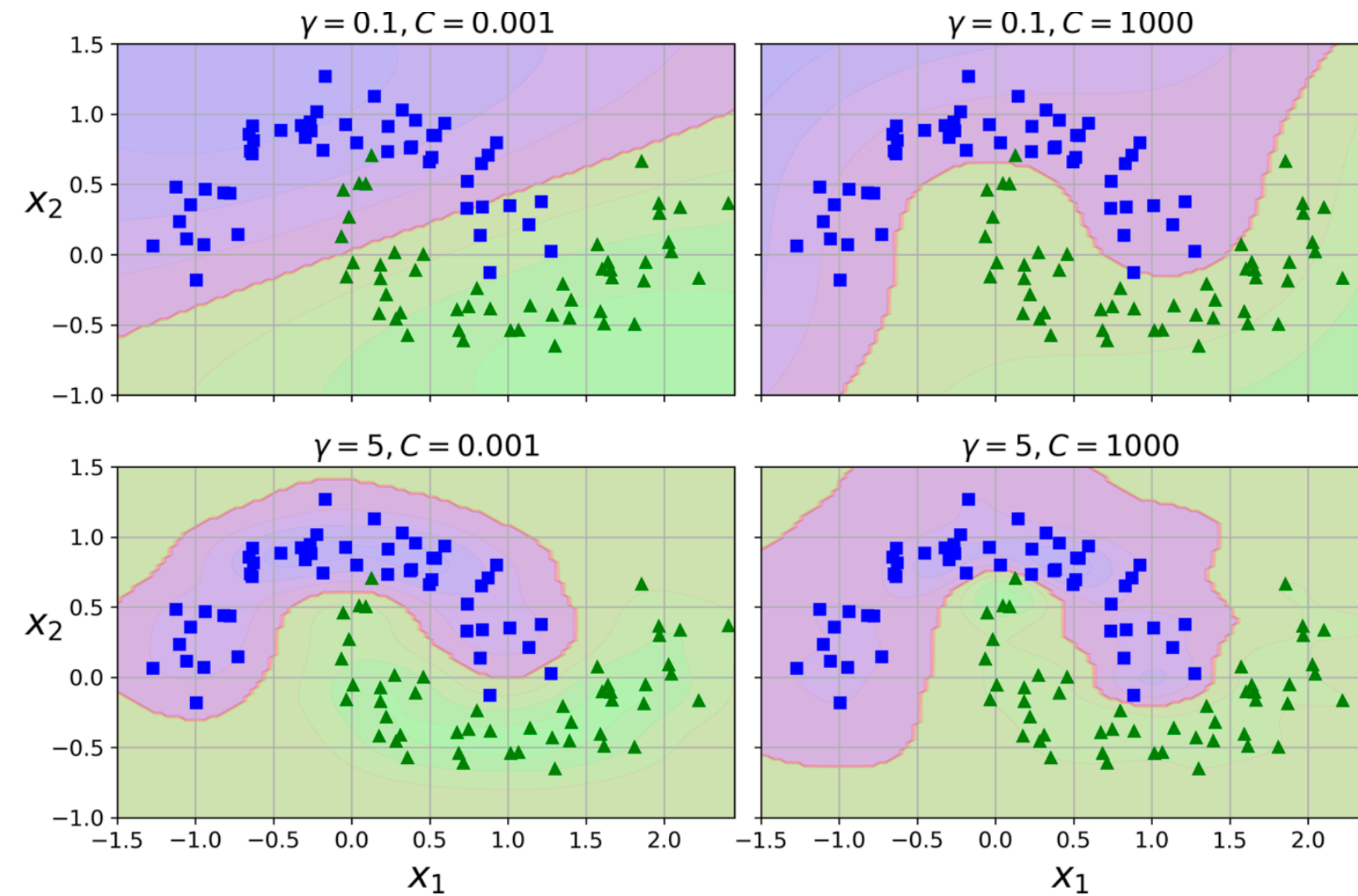
```
from sklearn.svm import SVC
poly_kernel_svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm_clf", SVC(kernel="poly", degree=3, coef0=1, C=5))
])
poly_kernel_svm_clf.fit(X, y)
```



SVM

- Kernel Gaussian RBF

```
rbf_kernel_svm_clf = Pipeline([  
    ("scaler", StandardScaler()),  
    ("svm_clf", SVC(kernel="rbf", gamma=5, C=0.001))  
])  
rbf_kernel_svm_clf.fit(X, y)
```



- **Vantagens**

- **Eficiência em Espaços de Alta Dimensão:** O SVM é eficaz em problemas com grandes dimensões de dados, onde outros algoritmos podem sofrer com a "maldição da dimensionalidade". Ele pode lidar bem com um número muito grande de características e ainda produzir modelos eficientes.
- **Robustez contra Overfitting:** O SVM tem uma boa capacidade de generalização, mesmo em conjuntos de dados pequenos ou com alto risco de overfitting. Isso ocorre porque ele tenta maximizar a margem entre as classes, o que contribui para modelos mais simples e com menos tendência a se ajustar excessivamente aos dados de treinamento.

- **Vantagens**

- **Capacidade de Trabalhar com Dados Não Lineares:** Através do *kernel trick*, o SVM pode lidar com dados não linearmente separáveis ao mapear as entradas para um espaço de características de maior dimensão, onde uma separação linear se torna possível. Isso permite que ele seja usado em uma ampla variedade de problemas complexos.
- **Versatilidade:** O SVM pode ser aplicado tanto para problemas de **classificação** quanto de **regressão**. Além disso, ele pode ser adaptado para trabalhar em tarefas multiclasse, embora o modelo básico seja binário. Essa flexibilidade o torna uma escolha atraente para uma grande variedade de aplicações.

- **Desvantagens**

- **Alta Complexidade Computacional em Grandes Conjuntos de Dados:** O SVM pode ser computacionalmente caro e lento, especialmente com grandes conjuntos de dados. O processo de treinamento envolve a resolução de um problema de otimização quadrática, o que pode ser lento quando o número de amostras é grande.
- **Escolha do hiperparâmetros de regularização e do kernel:** O desempenho do SVM depende fortemente da escolha adequada dos hiperparâmetros, como o de regularização C e o tipo de kernel. A escolha inadequada pode levar a um desempenho ruim. A busca por esses hiperparâmetros pode ser um processo demorado e requer uma validação cuidadosa.

- **Desvantagens**

- **Difícil Interpretação dos Modelos:** Os modelos gerados pelo SVM, especialmente quando se utiliza o *kernel trick* para transformações não lineares, podem ser difíceis de interpretar. Isso pode ser uma desvantagem em problemas onde a explicabilidade do modelo é importante.
- **Sensibilidade ao Ruído:** Embora o SVM seja robusto contra overfitting, ele pode ser sensível ao ruído, especialmente quando o parâmetro de regularização C é muito alto. Em cenários com muitos dados ruidosos ou sobrepostos, o SVM pode tentar ajustar o modelo com precisão excessiva aos dados ruidosos, levando a um desempenho ruim em dados inéditos.

Leitura adicional

- Machine Learning - A First Course for Engineers and Scientists

<http://smlbook.org/>

